

PRD: Doc-Helper

状态: Draft v0.2 目标: 先确认这个 Agent 产品的最小可用范围, 再进入开发, 开发一个最小 MVP。

项目定位

Doc-Helper 是一个**本地运行、浏览器访问**的文档处理助手, 面向"不想学 Office/WPS 复杂操作"的用户。任务完成有两种方式, 互为补充:

- 1. **直接调用工具**: 在界面选工具 → 上传文件 → 点运行 → 下载结果 (对小白最友好, 不依赖 LLM)
- 2. **智能体聊天**: 在侧边栏用自然语言描述需求, 智能体自动选择已有工具, 或现场**生成新工具** (草稿确认制)
- 3. **本地运行**: 文档不出本地, 不会发送数据泄密

三层架构 (详细规范见 [需求文档v1.md](#)) :



技术选型建议

层	选型	理由
后端	Node.js + Express	已装 Node 26; SSE 流式简单; spawn PowerShell 便捷
前端	原生 HTML/CSS/JS 单页 (无构建链)	自用工具, 避免打包流程; 文件上传、SSE 均有原生支持
工具执行	PowerShell 5.1 + powershell -File	工具脚本即 PowerShell, 与系统一致

层	选型	理由
文档处理	officecli (强制内置 vendor/officecli/)	唯一文档处理引擎，不依赖系统安装
LLM	OpenAI 兼容接口 (默认 DeepSeek)	支持 function calling; 厂商/base_url/key/模型均可界面配置

产品借鉴点

- **ChatGPT 式流式聊天**: 侧边栏聊天, SSE 逐字输出
- **工具市场 / 低代码平台**: 主界面工具卡片列表、启用/禁用开关、草稿标记
- **云存储上传体验**: 拖拽上传、执行进度、结果文件下载按钮
- **officecli 的 AI-friendly 设计**: schema 化能力描述, 可作为 system prompt 素材
- **应用市场审核 / 插件需启用机制**: agent 产出的工具默认草稿, 需手动启用后才生效

MVP 范围

In scope:

- 前端 3 界面: 主界面 (工具列表)、工具调用界面 (上传/运行/下载)、智能体聊天侧边栏 + LLM 配置入口
- 后端: 工具注册表、工具执行器 (超时/取消/JSON 解析)、LLM function calling 循环、agent 产出工具 (草稿确认制)、文件上传/结果下载
- 1~2个MVP工具

Out of scope (本期) :

- 多用户、账号体系、云部署、移动端、安装包
- pptx 处理
- 所见即所得的文档编辑

前端实现

页面架构总览

当前 PRD 定义为 1 个主界面, 两个子界面。

- 主界面: 包含工具列表, 用户在此管理与调用工具
 - 核心功能: 产品介绍, 工具列表, 等
- 工具调用界面: 包含工具信息, 上传文件的区域, 开始运行工具的按钮, 处理文档完毕后结果文档的下载按钮等。
 - 核心功能: 调用工具
- 智能体聊天侧边栏: 用户在此与智能体进行聊天, 智能体根据用户的需求在应用中添加/修改工具。
 - 用户添加厂商与大模型 API Key 的界面

关键用户链路

链路 1: 直接使用工具

1. 用户在主界面点击某工具卡片 (如 xlsx-merge-sheets)

2. 进入工具调用界面，查看工具说明与参数表单（文件类参数显示为上传区）
3. 上传待处理文件（拖拽 / 选择）
4. 点击"开始运行"，界面显示执行进度 / 日志
5. 处理完成，显示结果文件列表，点击"下载"

链路 2：智能体扩展工具

1. 用户打开聊天侧边栏："帮我做一个把多个 word 合并的工具"
2. 智能体生成 `tool.ps1` + `meta.json`，落盘为草稿 (`enabled=false`)
3. 侧边栏 / 主界面出现"新工具草稿"提示与"启用"按钮
4. 用户点击启用，工具出现在主界面列表，可被正常调用

链路 3：首次配置

1. 首次打开应用，进入侧边栏底部的"设置"
2. 选择厂商（默认 DeepSeek），填写 `base_url` / API Key / 模型名
3. 保存，配置持久化到本地 `config.json`
4. 未配置时聊天功能引导配置；工具直接调用不依赖 LLM，可正常使用

后端实现

模块划分

```
server/
├─ index.js      # Express 入口：静态托管 web/ + 挂载路由
├─ config.js     # 配置加载/保存（端口、LLM 配置）
├─ toolRegistry.js # 扫描 tools/*/meta.json，生成工具清单与 function schema
├─ toolRunner.js # 执行 tool.ps1: spawn、超时/取消、JSON 解析、日志
├─ fileStore.js  # 上传临时目录、任务结果目录、下载
├─ agent.js      # LLM 调用 + function calling 循环 + 工具生成（草稿）
└─ routes/
   ├─ tools.js   # 工具列表/详情/启用
   ├─ run.js     # 运行任务（创建/状态/取消）
   ├─ files.js   # 上传/下载
   └─ chat.js    # SSE 聊天流
```

关键设计点

- **工具执行安全**：只按工具名查表执行 `tools/` 下已注册工具；绝不执行 LLM 返回的任意字符串；路径做安全校验（防穿越）；超时 120s、可取消
- **OFFICECLI 注入**：执行工具时设置环境变量 `OFFICECLI=<vendor 路径>`，脚本统一 `& $env:OFFICECLI ...`
- **工具输出约定**：stdout 仅输出一个 JSON `{"ok":true,"outputFiles":[...],"summary":"..."}`；失败退出码非 0（详见 [需求文档v1.md](#) §5）
- **上传文件 → 工具参数映射**：`meta.json` 参数类型扩展约定——"`type`":"`file`" 的参数渲染为上传区，后端把上传文件放入任务临时目录并将目录路径作为参数值传入；`output` 类路径参数默认映射到任务结果目录，保证输出文件可下载

- **agent 生成工具**: LLM 输出脚本内容 → 后端校验 (文件名安全、防路径穿越) → 写入 `tools/<name>/` (`enabled=false`) → 前端提示启用
- **LLM 配置安全**: API key 保存于本地 `config.json` (加入 `.gitignore`) , 接口只返回"是否已配置", 不传完整 key
- **任务模型**: run 请求创建任务返回 `taskId`, 前端轮询或 SSE 获取进度/结果; 任务记录于内存 (MVP 重启即清)

接口草案

工具

方法	路径	说明
GET	/api/tools	工具列表 (含 enabled 状态、草稿标记)
GET	/api/tools/:name	工具详情 (含参数 schema)
POST	/api/tools/:name/enable	启用/禁用工具 <code>{"enabled": bool}</code>

运行与文件

方法	路径	说明
POST	/api/tools/:name/run	multipart: 上传文件 + 参数 JSON, 创建任务, 返回 <code>{taskId}</code>
GET	/api/tools/:name/run/:taskId	任务状态与结果 <code>{status, progress?, result?}</code>
POST	/api/tools/:name/run/:taskId/cancel	取消任务
GET	/api/files/:taskId/:file	下载结果文件
POST	/api/files/upload	上传待处理文件 (临时) → <code>{fileId, path}</code>

聊天 (SSE)

方法	路径	说明
POST	/api/chat	SSE 流事件: <code>text</code> (逐字)、 <code>tool_call</code> (执行工具)、 <code>tool_result</code> 、 <code>draft_tool</code> (新工具草稿)、 <code>done</code>

配置

方法	路径	说明
GET	/api/settings	读取配置 (key 脱敏, 仅返回是否已配置)
POST	/api/settings/llm	保存 <code>{vendor, baseUrl, apiKey, model}</code>
GET	/api/health	健康检查 (含 officecli 内置检测)

示例: 运行工具

```
POST /api/tools/xlsx-merge-sheets/run
Content-Type: multipart/form-data
files: [a.xlsx, b.xlsx]
json: {"inputDir": "@uploads", "output": "@results/merged.xlsx"} #
@uploads/@results 由后端映射

→ 200 {"taskId": "abc123"}

GET /api/tools/xlsx-merge-sheets/run/abc123
→ {"status": "running", "progress": "正在处理 a.xlsx"}
→ {"status": "done", "result": {"ok": true, "outputFiles": ["merged.xlsx"],
"summary": "已合并 2 个文件"}}

GET /api/files/abc123/merged.xlsx # 下载
```